```
import express from 'express';
import pg from 'pg';
import path from 'path';
const { DATABASE_URL, SCAN_KEY = '', PORT =
3000, SCAN_DAYS = '3' } = process.env;
if (!DATABASE_URL) {
console.error('DATABASE_URL is missing');
process.exit(1); }
const db = new pg.Pool({ connectionString:
DATABASE_URL, ssl: { rejectUnauthorized:
false }, max: 4 });
await db.query(`
CREATE TABLE IF NOT EXISTS jobs(
  id text PRIMARY KEY, dkey text UNIQUE, c
text, cat text, title text, company text,
city text,
  data jsonb NOT NULL, det timestamptz
DEFAULT now(), pub timestamptz, n int
DEFAULT 1);
CREATE INDEX IF NOT EXISTS jobs_det ON
jobs(det DESC);
CREATE INDEX IF NOT EXISTS jobs_cc ON
jobs(c, cat);
CREATE TABLE IF NOT EXISTS sources(name
text PRIMARY KEY, last_ok timestamptz,
last_err text, count int DEFAULT 0);`);

// ---------- Source: Bundesagentur für
Arbeit (public Jobsuche endpoint) ---------
-
```

```
const BASE =
'https://rest.arbeitsagentur.de/jobboerse/j
obsuche-service/pc/v4';
const HDR = { 'X-API-Key': 'jobboerse-
jobsuche' };
const QUERIES = [
  ...['Pflegefachkraft', 'Krankenpfleger',
'Altenpfleger', 'Pflegefachmann',
'Medizinische Fachangestellte'].map(was =>
({ was, cat: 'care', angebotsart: 1 })),
  ...['Photovoltaik', 'Solartechnik',
'Elektroniker', 'Elektroinstallateur',
'Elektriker'].map(was => ({ was, cat:
'solar', angebotsart: 1 })),
  ...['Pflege', 'Elektro', 'Solar',
'Photovoltaik'].map(was => ({ was, cat:
'ausb', angebotsart: 4 }))
];
const sleep = ms => new Promise(r =>
setTimeout(r, ms));
async function get(url) {
  for (let i = 0; i < 3; i++) {
    const r = await fetch(url, { headers:
HDR });
    if (r.status === 429) { await
sleep(1500 * (i + 1)); continue; }
    if (!r.ok) throw new Error('HTTP ' +
r.status);
    return r.json();
  }
  throw new Error('HTTP 429');
}
const dkey = (co, t, city) => (co + t +
city).toLowerCase().replace(/[^a-z0-
```

```
9äöüß]/g, '');
let scanning = false;
export async function scanBA() {
  if (scanning) return { skipped: true };
  scanning = true; let added = 0;
  try {
    for (const q of QUERIES) {
      const u = new URL(BASE + '/jobs');
      Object.entries({ was: q.was,
angebotsart: q.angebotsart,
veroeffentlichtseit: SCAN_DAYS, size: 50,
page: 1 }).forEach(([k, v]) =>
u.searchParams.set(k, v));
      const list = (await
get(u)).stellenangebote || [];
      for (const s of list) {
        const id = 'ba_' + s.refnr;
        if ((await db.query('SELECT 1 FROM
jobs WHERE id=$1', [id])).rowCount)
continue;
        if (added >= 80) break;
        let det = {};
        try { det = await
get(`${BASE}/jobdetails/${Buffer.from(s.ref
nr).toString('base64')}`); await
sleep(250); } catch { /* keep list data */
}
        const raw = JSON.stringify(det);
        const desc =
det.stellenangebotsBeschreibung ||
det.stellenbeschreibung || det.beschreibung
|| [s.titel,
s.beruf].filter(Boolean).join('. ');
        const email =
```

```
(raw.match(/[\w.+-]+@[\w-]+\.[\w.-]+/) ||
['']);[0];
        const city = s.arbeitsort?.ort ||
'', co = s.arbeitgeber || '';
        const url = s.externeUrl ||
`https://www.arbeitsagentur.de/jobsuche/job
detail/${encodeURIComponent(s.refnr)}`;
        const job = { t: s.titel || s.beruf
|| '', co, c: 'DE', city, cat: q.cat, desc:
String(desc).replace(/<[^>]+>/g, '
').slice(0, 6000), email, phone: '', sal:
'', dl: 0, url, src: 'Bundesagentur für
Arbeit' };
        const pub =
s.aktuelleVeroeffentlichungsdatum ||
s.modifikationsTimestamp || null;
        const r = await db.query(
          `INSERT INTO
jobs(id,dkey,c,cat,title,company,city,data,
pub) VALUES($1,$2,'DE',$3,$4,$5,$6,$7,$8)
           ON CONFLICT (dkey) DO UPDATE SET
n=jobs.n+1 RETURNING (xmax=0) AS fresh`,
          [id, dkey(co, job.t, city),
q.cat, job.t, co, city, job, pub]);
        if (r.rows[0].fresh) added++;
      }
    }
    await db.query(`INSERT INTO
sources(name,last_ok,last_err,count)
VALUES('Bundesagentur für
Arbeit',now(),NULL,(SELECT count(*) FROM
jobs WHERE id LIKE 'ba_%'))
      ON CONFLICT (name) DO UPDATE SET
last_ok=now(),last_err=NULL,count=EXCLUDED.
```

```
count`);
  } catch (e) {
    await db.query(`INSERT INTO
sources(name,last_err)
VALUES('Bundesagentur für Arbeit',$1) ON
CONFLICT (name) DO UPDATE SET last_err=$1`,
[String(e.message)]);
    console.error('scan error', e.message);
  } finally { scanning = false; }
  return { added };
}

// ---------- API ----------
const app = express();
app.get('/', (_q, res) =>
res.sendFile(path.resolve('index.html')));
app.get('/api/jobs', async (req, res) => {
  try {
    const { c, cat, q } = req.query, p =
[], w = [];
    if (c) { p.push(String(c));
w.push(`c=$${p.length}`); }
    if (cat) { p.push(String(cat));
w.push(`cat=$${p.length}`); }
    if (q) { p.push('%' + String(q) + '%');
w.push(`(title ILIKE $${p.length} OR
company ILIKE $${p.length} OR city ILIKE
$${p.length})`); }
    const lim = Math.min(+req.query.limit
|| 100, 300);
    const { rows } = await db.query(`SELECT
id,data,det,pub,n FROM jobs ${w.length ?
'WHERE ' + w.join(' AND ') : ''} ORDER BY
det DESC LIMIT ${lim}`, p);
```

```
    res.json(rows.map(r => ({ ...r.data,
id: r.id, det: +new Date(r.det), pub: r.pub
? +new Date(r.pub) : 0, n: r.n })));
  } catch (e) { res.status(500).json({
error: e.message }); }
});
app.get('/api/status', async (_q, res) => {
  try {
    const { rows } = await db.query('SELECT
name,last_ok,last_err,count FROM sources');
    const last = rows.reduce((m, r) =>
Math.max(m, r.last_ok ? +new
Date(r.last_ok) : 0), 0);
    res.json({ last, list: rows.map(r => ({
name: r.name, err: r.last_err, count:
r.count })) });
  } catch (e) { res.status(500).json({
error: e.message }); }
});
app.get('/api/scan', async (req, res) => {
  if (!SCAN_KEY || req.query.key !==
SCAN_KEY) return res.status(403).json({
error: 'forbidden' });
  res.json(await scanBA());
});
app.listen(PORT, () => {
console.log('JobRadar on', PORT); scanBA();
setInterval(scanBA, 20 * 60 * 1000); });
```